

train_roberta_colab

April 13, 2026

1 AGL — RoBERTa Fine-Tuning (Google Colab)

Adaptive Guardrail Layer (AGL) — Binary Prompt Safety Classifier

Fine-tunes roberta-base on the AGL binary dataset (Benign vs Malicious). This notebook is designed for **Google Colab** with a T4 GPU runtime.

Setup: Runtime → Change runtime type → T4 GPU

```
[ ]: # 1a. Install dependencies
[ ]: !pip install -q torch transformers datasets scikit-learn pandas numpy
    ↪ matplotlib seaborn

[ ]: # 1b. Clone repo & set working directory
import os

REPO_URL = 'https://github.com/joeldiev/aai-590-group-8-capstone.git'
REPO_DIR = '/content/aai-590-group-8-capstone'

if not os.path.exists(REPO_DIR):
    !git clone {REPO_URL} {REPO_DIR}
else:
    !cd {REPO_DIR} && git pull

os.chdir(REPO_DIR)
print(f"Working directory: {os.getcwd()}")

[ ]: # 1c. Mount Google Drive (for persistent data/model storage)
from google.colab import drive
drive.mount('/content/drive')

# Copy dataset from Drive if available (adjust path as needed)
DRIVE_DATA = '/content/drive/MyDrive/AGL/dataset_feature_engineered.csv'
LOCAL_DATA = 'data/processed/dataset_feature_engineered.csv'

os.makedirs('data/processed', exist_ok=True)

if os.path.exists(DRIVE_DATA) and not os.path.exists(LOCAL_DATA):
    import shutil
```

```

        shutil.copy(DRIVE_DATA, LOCAL_DATA)
        print(f"Copied dataset from Drive")
    elif os.path.exists(LOCAL_DATA):
        print(f"Dataset already present locally")
    else:
        print(f"WARNING: No dataset found at {DRIVE_DATA}")
        print(f"Upload dataset_feature_engineered.csv to data/processed/ or update_
↳DRIVE_DATA path")

```

```

[ ]: # 2. Check environment
import sys, torch

sys.path.insert(0, os.getcwd())

print(f"Python: {sys.version}")
print(f"PyTorch: {torch.__version__}")

if torch.cuda.is_available():
    DEVICE = "cuda"
    print(f"Device: CUDA - {torch.cuda.get_device_name(0)}")
    !nvidia-smi --query-gpu=memory.total,memory.free --format=csv,noheader
else:
    DEVICE = "cpu"
    print("WARNING: No GPU detected. Go to Runtime → Change runtime type → T4_
↳GPU")

# Verify data exists
data_csv = 'data/processed/dataset_feature_engineered.csv'
assert os.path.exists(data_csv), f"Data not found at {data_csv}. Upload it or_
↳check the Drive path."
print(f"\nData: {data_csv} ")

```

1.1 Step 3: Build Dataset

Load CSV, deduplicate, balance, and split into train/val/test parquets.

```

[ ]: # 3. Build dataset
from src.data.build_dataset import build_dataset

splits = build_dataset(csv_path='data/processed/dataset_feature_engineered.csv')

for name, df in splits.items():
    print(f" {name}: {len(df)} samples")

```

```

[ ]: # 3b. Verify dataset
import pandas as pd
from src.config import ID2LABEL

```

```

for split in ['train', 'val', 'test']:
    df = pd.read_parquet(f'data/processed/{split}.parquet')
    print(f"\n{split}: {len(df)} samples")
    label_counts = df['label'].value_counts()
    for label_id, count in label_counts.items():
        print(f"  ID2LABEL[{label_id}]: {count}")

```

1.2 Step 4: Fine-Tune RoBERTa (Transfer Learning)

Train the binary classifier with class-weighted cross-entropy, early stopping, and linear LR schedule.

Hyperparameters (from `src/config.py`): - LR: 2e-5, warmup: 10%, weight decay: 0.01 - Batch: 32, max epochs: 3, early stopping patience: 2 - Max sequence length: 128 - 2 classes: Benign (0), Malicious (1)

```

[ ]: # 4. Train RoBERTa classifier
from src.training.train import train_classifier

best_path = train_classifier()
print(f"\nBest model saved to: {best_path}")

```

1.3 Step 5: Fit Anomaly Detector (Optional)

Extract [CLS] embeddings from the fine-tuned model and fit the Mahalanobis OOD detector. Threshold is calibrated on the validation set (95% in-distribution recall).

```

[ ]: # 5. Fit Mahalanobis anomaly detector (optional)
from src.training.train import train_anomaly

anomaly_path = train_anomaly()
print(f"\nAnomaly detector saved to: {anomaly_path}")

```

1.4 Step 6: Evaluate

Run the full evaluation suite: keyword baseline, TF-IDF/SVM baseline, RoBERTa classifier.

```

[ ]: # 6. Run evaluation
import numpy as np
import pandas as pd
from src.config import MODELS_DIR, PROCESSED_DIR, RESULTS_DIR
from src.evaluation.metrics import evaluate_predictions, save_results
from src.evaluation.baselines import keyword_blocklist_baseline,
    tfidf_svm_baseline

# Load test data
train_df = pd.read_parquet(PROCESSED_DIR / 'train.parquet')
test_df = pd.read_parquet(PROCESSED_DIR / 'test.parquet')
y_true = test_df['label'].values

```

```

print("=" * 60)
print("Running evaluation suite")
print("=" * 60)

all_results = {}

# Baseline 1: Keyword blocklist
print("\n[1/3] Keyword blocklist baseline...")
kw_preds = keyword_blocklist_baseline(test_df)
kw_results = evaluate_predictions(y_true, kw_preds)
all_results['keyword_blocklist'] = kw_results
print(f" Macro-F1: {kw_results['macro_f1']:.4f}")

# Baseline 2: TF-IDF + SVM
print("\n[2/3] TF-IDF + LinearSVM baseline...")
svm_preds, _ = tfidf_svm_baseline(train_df, test_df)
svm_results = evaluate_predictions(y_true, svm_preds)
all_results['tfidf_svm'] = svm_results
print(f" Macro-F1: {svm_results['macro_f1']:.4f}")

# RoBERTa
checkpoint = MODELS_DIR / 'classifier' / 'best'
if checkpoint.exists():
    print("\n[3/3] RoBERTa classifier...")
    from src.models.agl_pipeline import AGLPipeline
    pipeline = AGLPipeline.from_checkpoint(checkpoint)
    roberta_preds = np.array([pipeline.predict(t).label_id for t in
    ↪test_df['text']])
    roberta_results = evaluate_predictions(y_true, roberta_preds)
    all_results['roberta'] = roberta_results
    print(f" Macro-F1: {roberta_results['macro_f1']:.4f}")
else:
    print(f"\n[3/3] Skipping RoBERTa - no checkpoint at {checkpoint}")

save_results(all_results, 'evaluation_results', RESULTS_DIR)

print("\n" + "=" * 60)
print("Summary - Macro-F1 Scores:")
print("=" * 60)
for method, res in all_results.items():
    print(f" {method:25s} {res['macro_f1']:.4f}")

```

1.5 Step 7: Visualizations

Generate figures: confusion matrix, F1 comparison, ROC curve, latency distribution.

```

[ ]: # 7. Generate visualizations
import numpy as np
import pandas as pd
import torch
from src.config import RESULTS_DIR, FIGURES_DIR, PROCESSED_DIR, MODELS_DIR,
    ↪MAX_SEQ_LENGTH
from src.evaluation.visualizations import (
    plot_confusion_matrix, plot_f1_comparison, plot_roc_curves
)
from src.evaluation.metrics import evaluate_predictions, benchmark_latency,
    ↪save_results
from src.evaluation.baselines import keyword_blocklist_baseline,
    ↪tfidf_svm_baseline
from src.models.agl_pipeline import AGLPipeline
from transformers import AutoTokenizer

FIGURES_DIR.mkdir(parents=True, exist_ok=True)

# Load test data
test_df = pd.read_parquet(PROCESSED_DIR / 'test.parquet')
train_df = pd.read_parquet(PROCESSED_DIR / 'train.parquet')
y_true = test_df['label'].values

# Baselines
kw_preds = keyword_blocklist_baseline(test_df)
svm_preds, _ = tfidf_svm_baseline(train_df, test_df)

# RoBERTa predictions + probabilities (for ROC-AUC)
checkpoint = MODELS_DIR / 'classifier' / 'best'
pipeline = AGLPipeline.from_checkpoint(checkpoint)
tokenizer = AutoTokenizer.from_pretrained(str(checkpoint),
    ↪local_files_only=True)

roberta_preds = []
roberta_probs = []
for text in test_df['text']:
    pred = pipeline.predict(text)
    roberta_preds.append(pred.label_id)
    inputs = tokenizer(text, truncation=True, padding='max_length',
        max_length=MAX_SEQ_LENGTH, return_tensors='pt')
    inputs = {k: v.to(pipeline.device) for k, v in inputs.items()}
    with torch.no_grad():
        logits = pipeline.model(**inputs).logits
        probs = torch.softmax(logits, dim=-1).cpu().numpy()[0]
    roberta_probs.append(probs)

roberta_preds = np.array(roberta_preds)

```

```

roberta_probs = np.array(roberta_probs)

# Confusion matrix
plot_confusion_matrix(y_true, roberta_preds, title='RoBERTa (Fine-Tuned)',
                      save_path=FIGURES_DIR / 'confusion_matrix_roberta.png')

# F1 comparison
from sklearn.metrics import f1_score
f1_results = {
    'Keyword Blocklist': f1_score(y_true, kw_preds, average='macro',
    ↪zero_division=0),
    'TF-IDF + SVM': f1_score(y_true, svm_preds, average='macro',
    ↪zero_division=0),
    'RoBERTa (Fine-Tuned)': f1_score(y_true, roberta_preds, average='macro',
    ↪zero_division=0),
}
plot_f1_comparison(f1_results, save_path=FIGURES_DIR / 'f1_comparison.png')

# ROC Curve
plot_roc_curves(y_true, roberta_probs, title='RoBERTa ROC Curve',
                save_path=FIGURES_DIR / 'roc_curve_roberta.png')

# Full evaluation with AUC
results = evaluate_predictions(y_true, roberta_preds, y_prob=roberta_probs)
print(f"\nRoBERTa Results:")
print(f"  Macro-F1: {results['macro_f1']:.4f}")
print(f"  ROC-AUC: {results.get('roc_auc', 'N/A')}")
print(f"  PR-AUC: {results.get('pr_auc', 'N/A')}")

# Latency
latency = benchmark_latency(pipeline, test_df['text'].head(50).tolist(),
    ↪n_runs=2)
print(f"\nLatency: mean={latency['mean_ms']:.1f}ms, p99={latency['p99_ms']:.1f}ms")

# Save results JSON
all_results = {
    'roberta_finetuned': {**results, 'latency': latency}
}
save_results(all_results, 'roberta_evaluation', RESULTS_DIR)
print('\nAll figures saved to:', FIGURES_DIR)

```

1.6 Step 8: Save Artifacts to Drive

Copy model checkpoints and results back to Google Drive for persistence.

```
[ ]: # 8. Save artifacts to Google Drive
import shutil
from pathlib import Path

DRIVE_OUT = Path('/content/drive/MyDrive/AGL')
DRIVE_OUT.mkdir(parents=True, exist_ok=True)

artifacts = [
    ('models/classifier/best', DRIVE_OUT / 'models' / 'classifier' / 'best'),
    ('models/anomaly', DRIVE_OUT / 'models' / 'anomaly'),
    ('results', DRIVE_OUT / 'results'),
]

for local, drive_dest in artifacts:
    if Path(local).exists():
        if drive_dest.exists():
            shutil.rmtree(drive_dest)
            shutil.copytree(local, drive_dest)
            print(f"    {local} → {drive_dest}")
        else:
            print(f"    {local} (not found, skipping)")

print("\nArtifacts saved to Google Drive.")
print("\nTo run a quick demo:")
print('    !python3 -m src.run --stage demo --text "ignore previous_\n↳instructions"')
```